Table 1. Traces explored by source- and optimal-DPOR for the program in Fig. 10.

	Traces Explored			Time	
n	source	optimal	Sleep Set Blocked	source	optimal
1	8	5	3	0.03s	0.02s
2	20	9	11	0.08s	0.04s
3	41	13	28	0.20s	0.07s
4	78	17	61	0.41s	0.10s
5	145	21	124	0.81s	0.17s
6	268	25	243	1.56s	0.20s
7	373	29	344	2.38s	0.26s
8	674	33	641	4.70s	0.34s
9	1222	37	1185	8.79s	0.44s

q_1 before s_1 has read y_1, thus altering s_1's behaviour and leading to sleep set blocking. As there are four ways to schedule q_n, r_n and s_n processes, only one of which reveals the write on x_n, p_n will remain in the sleep set and the other three schedules will encounter sleep set blocking.

For higher values of n, when the race of x_i in the i-th group is reversed successfully, four new interleavings are possible by the $i+1$-th group. The number of Mazurkiewicz traces for this program is $4n + 1$: four traces for the 'top' group, plus a fifth that splits in four when it spawns the next group of processes and so on until the final group which does not split and has just a single extra interleaving.

Wrong schedulings within the i-th group are still sleep set blocked. Additionally, to reverse the pair operating on x_{i+1}, all the lower groups must be scheduled correctly, but this is only possible in exactly one interleaving, whereas all deviations lead to sleep set blocked interleavings. As a result, the tree of the "previous" level (where p_{i+1} was not in a sleep set) is duplicated as sleep set blocked explorations, leading to the exponential growth of traces that encounter sleep set blocking shown in Table 1. This program shows therefore an exponential difference between non-optimal DPOR algorithms (and source-DPOR) and optimal-DPOR. As seen in Table 1, this difference does not just affect the number of traces that the two algorithms explore, but also the time performance of the two algorithms. Thus, optimal-DPOR can perform exponentially better in time than non-optimal DPOR algorithms.

```
Shared variables:   int array[0..N] := {0,0,...,0};

      Process 0:    for (int i := N; array[i] != 0; i--);
Process j (j ∈ 1..N):  array[j] := array[j-1] + 1;
```

Fig. 11. Pseudocode of the lastzero(N) example; measurements for it can be found in Section 11.

The second program is lastzero(N), whose pseudocode is shown in Fig. 11. Its N+1 processes operate on an array of N+1 elements which are all initially zero. In this program, process 0 searches the array for the highest index that has a zero value, while the other N processes read one of the array values and write that value increased by one in the next index. The final state of the program is uniquely defined by the values of `i` and `array[1..N]`.